

```
!pip install -q kagglehub tensorflow scikit-learn pandas numpy matplotlib seaborn
```

```
import kagglehub
```

```
# Download latest version
```

```
path = kagglehub.dataset_download("sumaya23abdul/automobile-database")
```

```
print("Path to dataset files:", path)
```

```
Downloading to /root/.cache/kagglehub/datasets/sumaya23abdul/automobile-database/1.archive...
```

```
100%|██████████| 4.87k/4.87k [00:00<00:00, 12.1MB/s]Extracting files...
```

```
Path to dataset files: /root/.cache/kagglehub/datasets/sumaya23abdul/automobile-database/versions/1
```

```
import os
```

```
print("Files inside dataset folder:")
```

```
for file in os.listdir(path):  
    print(file)
```

```
Files inside dataset folder:  
Automobile_data.csv
```

```
import pandas as pd
```

```
import os
```

```
csv_files = [  
    os.path.join(path, file)  
    for file in os.listdir(path)  
    if file.lower().endswith(".csv")  
]
```

```
print("CSV files found:")  
for file in csv_files:  
    print(file)
```

```
CSV files found:  
/root/.cache/kagglehub/datasets/sumaya23abdul/automobile-database/versions/1/Automobile_data.csv
```

```
df = pd.read_csv(csv_files[0])
```

```
print("Dataset shape:", df.shape)  
df.head()
```

```
Dataset shape: (205, 26)
```

	symboling	normalized- losses	make	fuel- type	aspiration	num- of- doors	body- style	drive- wheels	engine- location	wheel- base	...
0	3	?	alfa-romero	gas	std	two	convertible	rwd	front	88.6	...
1	3	?	alfa-romero	gas	std	two	convertible	rwd	front	88.6	...
2	1	?	alfa-romero	gas	std	two	hatchback	rwd	front	94.5	...
3	2	164	audi	gas	std	four	sedan	fwd	front	99.8	...
4	2	164	audi	gas	std	four	sedan	4wd	front	99.4	...

```
5 rows × 26 columns
```

```
print("Columns:")
print(df.columns.tolist())
```

```
print("\nDataset information:")
df.info()
```

Columns:

['symboling', 'normalized-losses', 'make', 'fuel-type', 'aspiration', 'num-of-doors', 'body-style',

Dataset information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 205 entries, 0 to 204

Data columns (total 26 columns):

#	Column	Non-Null Count	Dtype
0	symboling	205 non-null	int64
1	normalized-losses	205 non-null	object
2	make	205 non-null	object
3	fuel-type	205 non-null	object
4	aspiration	205 non-null	object
5	num-of-doors	205 non-null	object
6	body-style	205 non-null	object
7	drive-wheels	205 non-null	object
8	engine-location	205 non-null	object
9	wheel-base	205 non-null	float64
10	length	205 non-null	float64
11	width	205 non-null	float64
12	height	205 non-null	float64
13	curb-weight	205 non-null	int64
14	engine-type	205 non-null	object
15	num-of-cylinders	205 non-null	object
16	engine-size	205 non-null	int64
17	fuel-system	205 non-null	object
18	bore	205 non-null	object
19	stroke	205 non-null	object
20	compression-ratio	205 non-null	float64
21	horsepower	205 non-null	object
22	peak-rpm	205 non-null	object
23	city-mpg	205 non-null	int64
24	highway-mpg	205 non-null	int64
25	price	205 non-null	object

dtypes: float64(5), int64(5), object(16)

memory usage: 41.8+ KB

```
print(df.isnull().sum())
```

symboling	0
normalized-losses	0
make	0
fuel-type	0
aspiration	0
num-of-doors	0
body-style	0
drive-wheels	0
engine-location	0
wheel-base	0
length	0
width	0
height	0
curb-weight	0
engine-type	0
num-of-cylinders	0
engine-size	0
fuel-system	0
bore	0
stroke	0
compression-ratio	0
horsepower	0
peak-rpm	0
city-mpg	0
highway-mpg	0
price	0
dtype: int64	

```
numeric_columns = df.select_dtypes(include=["number"]).columns
```

```
print("Numerical columns:")  
print(numeric_columns.tolist())
```

```
Numerical columns:  
['symboling', 'wheel-base', 'length', 'width', 'height', 'curb-weight', 'engine-size', 'compression-
```

```
numeric_columns = df.select_dtypes(include=["number"]).columns.tolist()
```

```
print("Available numerical columns:")  
print(numeric_columns)
```

```
Available numerical columns:  
['symboling', 'wheel-base', 'length', 'width', 'height', 'curb-weight', 'engine-size', 'compression-
```

```
features = numeric_columns[:3]
```

```
print("Selected features:")  
print(features)
```

```
data = df[features].copy()
```

```
data.head()
```

```
Selected features:  
['symboling', 'wheel-base', 'length']
```

	symboling	wheel-base	length	
0	3	88.6	168.8	
1	3	88.6	168.8	
2	1	94.5	171.2	
3	2	99.8	176.6	
4	2	99.4	176.6	

Next steps: [Generate code with data](#) [New interactive sheet](#)

```
import numpy as np  
data = data.replace("?", np.nan)  
  
for column in data.columns:  
    data[column] = pd.to_numeric(data[column], errors="coerce")  
  
data = data.fillna(data.median())  
  
print(data.isnull().sum())
```

```
symboling      0  
wheel-base    0  
length         0  
dtype: int64
```

```
from sklearn.preprocessing import MinMaxScaler  
  
scaler = MinMaxScaler()  
  
scaled_data = scaler.fit_transform(data)  
  
print("Scaled data shape:", scaled_data.shape)  
print(scaled_data[:5])
```

```
Scaled data shape: (205, 3)
[[1.          0.05830904 0.41343284]
 [1.          0.05830904 0.41343284]
 [0.6         0.2303207  0.44925373]
 [0.8         0.38483965 0.52985075]
 [0.8         0.37317784 0.52985075]]
```

```
import numpy as np

time_steps = 5

X = []
y = []

for i in range(len(scaled_data) - time_steps):
    X.append(scaled_data[i:i + time_steps])
    y.append(scaled_data[i + time_steps, 0])

X = np.array(X)
y = np.array(y)

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (200, 5, 3)
y shape: (200,)
```

```
#1. Basic / Vanilla LSTM
import tensorflow as tf

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

print("TensorFlow version:", tf.__version__)
```

```
TensorFlow version: 2.20.0
```

```
basic_model = Sequential([
    LSTM(50, input_shape=(X.shape[1], X.shape[2])),
    Dense(1)
])

basic_model.compile(
    optimizer="adam",
    loss="mse"
)

basic_model.summary()
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an
super().__init__(**kwargs)
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 50)	10,800
dense (Dense)	(None, 1)	51

```
Total params: 10,851 (42.39 KB)
Trainable params: 10,851 (42.39 KB)
Non-trainable params: 0 (0.00 B)
```

```
history_basic = basic_model.fit(
    X,
    y,
    epochs=20,
    batch_size=16,
```

```
validation_split=0.2,  
verbose=0  
)
```

```
prediction = basic_model.predict(X[-1].reshape(1, X.shape[1], X.shape[2]))  
  
print("Predicted value:", prediction[0][0])
```

1/1 ————— 0s 106ms/step
Predicted value: 0.38459083

```
#stacked  
stacked_model = Sequential([  
    LSTM(  
        64,  
        return_sequences=True,  
        input_shape=(X.shape[1], X.shape[2])  
    ),  
  
    LSTM(32),  
  
    Dense(1)  
)  
  
stacked_model.compile(  
    optimizer="adam",  
    loss="mse"  
)  
  
stacked_model.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
lstm_7 (LSTM)	(None, 5, 64)	17,408
lstm_8 (LSTM)	(None, 32)	12,416
dense_4 (Dense)	(None, 1)	33

Total params: 29,857 (116.63 KB)
Trainable params: 29,857 (116.63 KB)

```
history_stacked = stacked_model.fit(  
    X,  
    y,  
    epochs=20,  
    batch_size=16,  
    validation_split=0.2,  
    verbose=0  
)
```

```
stacked_prediction = stacked_model.predict(  
    X[-1].reshape(1, X.shape[1], X.shape[2])  
)  
  
print("Stacked LSTM prediction:", stacked_prediction[0][0])
```

1/1 ————— 0s 164ms/step
Stacked LSTM prediction: 0.40896097

```
#bidirectional  
from tensorflow.keras.layers import Bidirectional  
  
bidirectional_model = Sequential([  
    Bidirectional(  
        LSTM(64,  
            return_sequences=True,  
            input_shape=(X.shape[1], X.shape[2])  
        )  
    )  
)
```

```

        LSTM(50),
        input_shape=(X.shape[1], X.shape[2])
    ),

    Dense(1)
])

bidirectional_model.compile(
    optimizer="adam",
    loss="mse"
)

bidirectional_model.summary()

```

Model: "sequential_6"

Layer (type)	Output Shape	Param #
bidirectional_1 (Bidirectional)	(None, 100)	21,600
dense_6 (Dense)	(None, 1)	101

Total params: 21,701 (84.77 KB)
 Trainable params: 21,701 (84.77 KB)

```

history_bidirectional = bidirectional_model.fit(
    X,
    y,
    epochs=20,
    batch_size=16,
    validation_split=0.2,
    verbose=0
)

```

```

bi_prediction = bidirectional_model.predict(
    X[-1].reshape(1, X.shape[1], X.shape[2])
)

print("Bidirectional prediction:", bi_prediction[0][0])

```

1/1 ————— 0s 252ms/step
 Bidirectional prediction: 0.37674046

```

#multivariate
multivariate_model = Sequential([
    LSTM(
        64,
        input_shape=(X.shape[1], X.shape[2])
    ),

    Dense(1)
])

multivariate_model.compile(
    optimizer="adam",
    loss="mse"
)

multivariate_model.summary()

```

Model: "sequential_8"

Layer (type)	Output Shape	Param #
lstm_12 (LSTM)	(None, 64)	17,408
dense_8 (Dense)	(None, 1)	65

Total params: 17,473 (68.25 KB)

Trainable params: 17,473 (68.25 KB)

```
history_multivariate = multivariate_model.fit(
    X,
    y,
    epochs=20,
    batch_size=16,
    validation_split=0.2,
    verbose=0
)
```

```
#many_to_one_model
many_to_one_model = Sequential([
    LSTM(
        50,
        input_shape=(X.shape[1], X.shape[2])
    ),
    Dense(1)
])

many_to_one_model.compile(
    optimizer="adam",
    loss="mse"
)

many_to_one_model.summary()
```

Model: "sequential_11"

Layer (type)	Output Shape	Param #
lstm_15 (LSTM)	(None, 50)	10,800
dense_11 (Dense)	(None, 1)	51

Total params: 10,851 (42.39 KB)

Trainable params: 10,851 (42.39 KB)

```
history_many_to_one = many_to_one_model.fit(
    X,
    y,
    epochs=20,
    batch_size=16,
    validation_split=0.2,
    verbose=0
)
```

```
#many to many
X_mtm = []
y_mtm = []

for i in range(len(scaled_data) - time_steps):
    X_mtm.append(
        scaled_data[i:i + time_steps]
    )

    y_mtm.append(
```

```

        scaled_data[i:i + time_steps, 0].reshape(-1, 1)
    )

X_mtm = np.array(X_mtm)
y_mtm = np.array(y_mtm)

print("X_mtm shape:", X_mtm.shape)
print("y_mtm shape:", y_mtm.shape)

```

```

X_mtm shape: (200, 5, 3)
y_mtm shape: (200, 5, 1)

```

```

many_to_many_model = Sequential([
    LSTM(
        64,
        return_sequences=True,
        input_shape=(X_mtm.shape[1], X_mtm.shape[2])
    ),

    Dense(1)
])

many_to_many_model.compile(
    optimizer="adam",
    loss="mse"
)

many_to_many_model.summary()

```

Model: "sequential_12"

Layer (type)	Output Shape	Param #
lstm_16 (LSTM)	(None, 5, 64)	17,408
dense_12 (Dense)	(None, 5, 1)	65

```

Total params: 17,473 (68.25 KB)
Trainable params: 17,473 (68.25 KB)

```

```

history_many_to_many = many_to_many_model.fit(
    X_mtm,
    y_mtm,
    epochs=20,
    batch_size=16,
    validation_split=0.2,
    verbose=0
)

```

```

mtm_prediction = many_to_many_model.predict(
    X_mtm[-1].reshape(1, X_mtm.shape[1], X_mtm.shape[2])
)

print("Prediction shape:", mtm_prediction.shape)

```

```

1/1 ————— 0s 102ms/step
Prediction shape: (1, 5, 1)

```

```

#autoencoder
from tensorflow.keras.layers import RepeatVector, TimeDistributed

autoencoder = Sequential([

    # Encoder
    LSTM(
        64,
        input_shape=(X.shape[1], X.shape[2])
    )

```

```

    ),

    # Repeat encoded vector
    RepeatVector(X.shape[1]),

    # Decoder
    LSTM(
        64,
        return_sequences=True
    ),

    # Output
    TimeDistributed(
        Dense(X.shape[2])
    )
])

autoencoder.compile(
    optimizer="adam",
    loss="mse"
)

autoencoder.summary()

```

Model: "sequential_14"

Layer (type)	Output Shape	Param #
lstm_19 (LSTM)	(None, 64)	17,408
repeat_vector_1 (RepeatVector)	(None, 5, 64)	0
lstm_20 (LSTM)	(None, 5, 64)	33,024
time_distributed_1 (TimeDistributed)	(None, 5, 3)	195

Total params: 50,627 (197.76 KB)
 Trainable params: 50,627 (197.76 KB)

```

history_autoencoder = autoencoder.fit(
    X,
    X,
    epochs=20,
    batch_size=16,
    validation_split=0.2,
    verbose=0
)

```

```

reconstructed = autoencoder.predict(
    X[-1].reshape(1, X.shape[1], X.shape[2])
)

print("Original shape:", X[-1].shape)
print("Reconstructed shape:", reconstructed.shape)

```

WARNING:tensorflow:5 out of the last 5 calls to <function TensorFlowTrainer.make_predict_function.<1
 1/1 ————— 0s 226ms/step
 Original shape: (5, 3)
 Reconstructed shape: (1, 5, 3)

```

# Text / Next-Word LSTM
from tensorflow.keras.layers import Embedding

vocab_size = 5000

text_model = Sequential([
    Embedding(

```

```

        input_dim=vocab_size,
        output_dim=64
    ),

    LSTM(64),

    Dense(
        vocab_size,
        activation="softmax"
    )
])

text_model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy"
)

text_model.summary()

```

Model: "sequential_16"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	?	0 (unbuilt)
lstm_22 (LSTM)	?	0 (unbuilt)
dense_16 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)

```

#Dropout
from tensorflow.keras.layers import Dropout

dropout_model = Sequential([
    LSTM(
        64,
        input_shape=(X.shape[1], X.shape[2])
    ),

    Dropout(0.2),

    Dense(1)
])

dropout_model.compile(
    optimizer="adam",
    loss="mse"
)

dropout_model.summary()

```

Model: "sequential_18"

Layer (type)	Output Shape	Param #
lstm_24 (LSTM)	(None, 64)	17,408
dropout_1 (Dropout)	(None, 64)	0
dense_18 (Dense)	(None, 1)	65

Total params: 17,473 (68.25 KB)
Trainable params: 17,473 (68.25 KB)
Non-trainable params: 0 (0.00 B)

```

history_dropout = dropout_model.fit(
    X,
    y,
    epochs=20,

```

```
batch_size=16,  
validation_split=0.2,  
verbose=0  
)
```

```
last_sequence = X[-1]  
  
last_sequence = last_sequence.reshape(  
    1,  
    X.shape[1],  
    X.shape[2]  
)  
  
prediction = dropout_model.predict(  
    last_sequence  
)  
  
print("Predicted scaled value:", prediction[0][0])
```

1/1 ————— 0s 37ms/step
Predicted scaled value: 0.3813738

```
dummy = np.zeros((1, len(features)))  
  
dummy[0, 0] = prediction[0][0]  
  
original_value = scaler.inverse_transform(dummy)  
  
print(  
    "Predicted original-scale value:",  
    original_value[0, 0]  
)
```

Predicted original-scale value: -0.09313103556633007

```
dropout_model.save("automobile_lstm_model.keras")  
  
print("Model saved successfully!")
```

Model saved successfully!